Proyecto: Alke Wallet – ABP 7

1. Situación Inicial

La empresa fintech Alke Financial requiere el desarrollo de una aplicación web que permita gestionar la información de usuarios y realizar operaciones básicas utilizando una base de datos relacional.

Para ello, el equipo de desarrollo debe implementar una solución utilizando Django como framework principal, integrándolo con una base de datos mediante el ORM (Object Relational Mapping). Esto permitirá definir modelos de datos, aplicar migraciones y realizar operaciones de consulta y manipulación de información de forma estructurada.

El proyecto se desarrolla en el entorno de programación Visual Studio Code, utilizando herramientas gratuitas y asegurando la correcta persistencia de los datos, así como una integración coherente entre modelos, vistas y templates dentro de la arquitectura del framework.

2. Objetivo del Proyecto

Desarrollar una aplicación web funcional que permita almacenar, consultar y gestionar información persistente mediante el uso del ORM de Django, aplicando migraciones y operaciones CRUD (Create, Read, Update, Delete), garantizando la integridad, consistencia y correcta gestión de los datos en la base de datos.

3. Requerimientos del Proyecto

3.1 Base de Datos y ORM

- Configurar la base de datos utilizando SQLite.
- Definir modelos representativos mediante el ORM de Django.
- Establecer relaciones entre los diferentes modelos del sistema.

3.2 Migraciones

- Generar y aplicar migraciones correctamente utilizando las herramientas de Django.
- Mantener sincronizado el esquema de la base de datos con los modelos definidos en la aplicación.

3.3 Operaciones CRUD

- Implementar operaciones de creación, lectura, actualización y eliminación de registros mediante el ORM.
- Integrar las operaciones CRUD dentro de las vistas y templates para permitir la interacción del usuario con el sistema.

4. Modelos Implementados

Usuario (User de Django)

- Se utiliza el modelo de usuario nativo de Django para gestionar la autenticación y administración de usuarios dentro del sistema.

Balance

Representa el saldo disponible de cada usuario dentro de la billetera digital.

Campos principales:

- usuario → OneToOneField con el modelo User
- monto → DecimalField que almacena el saldo disponible

Contacto

Representa los contactos registrados por cada usuario, utilizados para facilitar transferencias de dinero dentro de la aplicación.

Campos principales:

- usuario → ForeignKey que identifica al propietario del contacto
- nombre
- documento
- email
- cuenta
- tipo_cuenta
- banco

Transaccion

Registra los movimientos de dinero realizados dentro del sistema, permitiendo llevar un historial de operaciones.

- Campos principales:
- tipo → tipo de operación (depósito, retiro o transferencia)
- monto → monto de la transacción
- emisor → usuario que envía el dinero
- receptor → usuario que recibe el dinero
- fecha → fecha y hora en que se registra la operación

5. Relaciones Principales

Las relaciones entre los modelos del sistema se definen de la siguiente manera:

- Un usuario tiene un único balance (relación 1:1).
- Un usuario puede registrar múltiples contactos (relación 1:N).
- Un usuario puede enviar múltiples transacciones (relación 1:N).
- Un usuario puede recibir múltiples transacciones (relación 1:N).

Estas relaciones permiten mantener una estructura de datos coherente para el funcionamiento de la billetera digital y el registro de las operaciones realizadas por los usuarios.

6. Arquitectura del Sistema

La aplicación Alke Wallet fue desarrollada utilizando el framework Django, el cual sigue una arquitectura basada en el patrón MVT (Model – View – Template).

Este patrón permite separar la lógica del sistema en diferentes componentes para facilitar su mantenimiento y escalabilidad.

Model (Modelo)

Los modelos representan la estructura de la base de datos y se encargan de definirlas entidades del sistema y sus relaciones.

En este proyecto los modelos principales son:

- User: modelo de autenticación de Django.
- Balance: representa el saldo disponible de cada usuario.
- Contacto: almacena los contactos registrados por cada usuario.
- Transaccion: registra los movimientos financieros realizados en la billetera.

Los modelos se comunican con la base de datos utilizando el ORM de Django, permitiendo realizar consultas y operaciones sin necesidad de escribir SQL directamente.

View (Vista)

Las vistas contienen la lógica de negocio de la aplicación y permiten procesar las solicitudes del usuario.

Entre las principales funcionalidades implementadas se encuentran:

- Autenticación de usuarios (login y logout)
- Consulta del saldo disponible
- Registro y administración de contactos
- Envío de dinero entre usuarios
- Registro automático de transacciones

Las vistas interactúan con los modelos utilizando el ORM para consultar, crear o modificar información en la base de datos.

Template (Plantillas)

Los templates corresponden a la interfaz visual del sistema, construida utilizando HTML, CSS y Bootstrap.

Permiten mostrar al usuario:

- Su saldo disponible
- El historial de transacciones
- La lista de contactos registrados
- Formularios para realizar transferencias

Los templates reciben datos desde las vistas y los presentan dinámicamente utilizando el motor de plantillas de Django.

7. Flujo de Operación del Sistema

El funcionamiento de la billetera digital sigue el siguiente flujo general:

1. Autenticación del usuario

El usuario accede al sistema mediante su nombre de usuario y contraseña, utilizando el sistema de autenticación integrado de Django.

2. Consulta de saldo

Una vez autenticado, el usuario puede visualizar su saldo disponible, el cual se encuentra almacenado en el modelo Balance.

3. Gestión de contactos

El usuario puede registrar contactos dentro del sistema para facilitar futuras transferencias. Cada contacto queda asociado al usuario que lo creó.

4. Envío de dinero

Para realizar una transferencia el sistema ejecuta los siguientes pasos:

- El usuario selecciona un contacto registrado.
- Ingresa el monto a transferir.
- El sistema valida que el monto sea mayor que cero.
- Se verifica que el usuario tenga saldo suficiente.
- Se descuenta el monto del balance del emisor.
- Se suma el monto al balance del receptor.
- Se registra la operación en el modelo **Transaccion**.

5. Registro del historial de transacciones

Cada operación realizada queda almacenada en la tabla Transaccion, permitiendo que el usuario visualice su historial de movimientos, indicando:

- Tipo de operación
- Monto
- Fecha
- Usuario emisor o receptor

8. Conclusión

El proyecto Alke Wallet permite implementar una billetera digital básica utilizando el framework Django y su ORM para gestionar datos de forma estructurada y segura.

Mediante el uso de modelos, migraciones y operaciones CRUD, se logró desarrollar una aplicación funcional que permite administrar usuarios, contactos y transacciones financieras, manteniendo la integridad de la información almacenada en la base de datos.

Este proyecto demuestra la capacidad de Django para construir aplicaciones web escalables y seguras, integrando lógica de negocio, persistencia de datos y presentación de información dentro de una arquitectura organizada.

12. Posibles Mejoras Futuras del Sistema

Si bien el sistema **Alke Wallet** cumple con los requerimientos planteados para este proyecto, existen diversas mejoras que podrían implementarse en futuras versiones con el fin de ampliar sus funcionalidades, mejorar la seguridad y optimizar la experiencia del usuario.

12.1 Seguridad avanzada

Implementar mecanismos de seguridad adicionales para proteger las cuentas de los usuarios, tales como:

Autenticación en dos factores (2FA).

- Verificación de correo electrónico al momento de registrarse.
- Encriptación adicional de información sensible.

12.2 Historial detallado de transacciones

Mejorar el sistema de historial de movimientos permitiendo:

- Filtrar transacciones por fecha.
- Buscar transacciones por tipo o usuario.
- Exportar el historial en formatos como **PDF o CSV**.

12.3 Notificaciones de operaciones

Agregar un sistema de notificaciones que informe al usuario cuando se realice una operación en su cuenta, por ejemplo:

- Notificación al recibir dinero.
- Confirmación de transferencias realizadas.
- Alertas de seguridad.
- Estas notificaciones podrían enviarse mediante:
- Correo electrónico.
- Notificaciones dentro de la aplicación.

12.4 Mejora en la gestión de contactos

Ampliar la funcionalidad del módulo de contactos permitiendo:

- Edición de información de contactos.
- Agrupación de contactos frecuentes.
- Validación automática de cuentas bancarias.

12.5 Integración con APIs externas

En una versión más avanzada del sistema se podría integrar la aplicación con **APIs de instituciones financieras**, lo que permitiría:

- Validar cuentas bancarias.
- Simular transferencias reales.
- Obtener información de bancos asociados.

12.6 Interfaz de usuario mejorada

Mejorar la experiencia visual de la aplicación mediante:

- Dashboards con **gráficos de movimientos financieros**.
- Visualización más intuitiva del saldo y movimientos.

12.7 Escalabilidad de la base de datos

En entornos de producción se podría migrar la base de datos desde **SQLite a PostgreSQL**, lo cual permitiría:

- Manejar mayor volumen de usuarios.
- Mejorar el rendimiento del sistema.
- Implementar funcionalidades avanzadas de base de datos.

12.8 Aplicación móvil

Como evolución del proyecto, se podría desarrollar una **aplicación móvil** que permita a los usuarios acceder a la billetera digital desde sus dispositivos móviles, utilizando tecnologías como:

DB Browser for SQLite - C:\Users\kgarc\OneDrive\Escritorio\alkewallet\alkeweb\db.sqlite3

File Edit View Tools Help

Open Project | Save Project | Attach Database | Close Database

Browse Data | Edit Pragmas | Execute SQL | Database Structure

Create Table | Create Index | Modify Table | Delete Table | Print | Refresh

Name	Type	Schema
Tables (14)		
auth_group		CREATE TABLE "auth_group" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "name" varchar(150) NOT NULL U
auth_group_permissions		CREATE TABLE "auth_group_permissions" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "group_id" integer N
auth_permission		CREATE TABLE "auth_permission" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "content_type_id" integer NO
auth_user		CREATE TABLE "auth_user" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "password" varchar(128) NOT NUL
auth_user_groups		CREATE TABLE "auth_user_groups" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "user_id" integer NOT NUL
auth_user_user_permissions		CREATE TABLE "auth_user_user_permissions" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "user_id" integer
core_balance		CREATE TABLE "core_balance" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "usuario_id" integer NOT NULL
core_contacto		CREATE TABLE "core_contacto" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "nombre" varchar(100) NOT NU
core_transaccion		CREATE TABLE "core_transaccion" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "tipo" varchar(20) NOT NUL
django_admin_log		CREATE TABLE "django_admin_log" ("id" integer NOT NULL PR
django_content_type		CREATE TABLE "django_content_type" ("id" integer NOT NULL
django_migrations		CREATE TABLE "django_migrations" ("id" integer NOT NULL PR
django_session		CREATE TABLE "django_session" ("session_key" varchar(40) NO
sqlite_sequence		CREATE TABLE sqlite_sequence(name,seq)
Indices (20)		
auth_group_permissions_group_id_b1...		CREATE INDEX "auth_group_permissions_group_id_b120cbf9" ON "auth_group_permissions" ("group_id")
auth_group_permissions_group_id_per...		CREATE UNIQUE INDEX "auth_group_permissions_group_id_permission_id_0cd325b0_uniq" ON "auth_group_permission
auth_group_permissions_permission_id...		CREATE INDEX "auth_group_permissions_permission_id_84c5c92e" ON "auth_group_permissions" ("permission_id")
auth_permission_content_type_id_2f47...		CREATE INDEX "auth_permission_content_type_id_2f476e4b" ON "auth_permission" ("content_type_id")
auth_permission_content_type_id_cod...		CREATE UNIQUE INDEX "auth_permission_content_type_id_codename_01ab375a_uniq" ON "auth_permission" ("content
auth_user_groups_group_id_97559544		CREATE INDEX "auth_user_groups_group_id_97559544" ON "auth_user_groups" ("group_id")
auth_user_groups_user_id_6a12ed8b		CREATE INDEX "auth_user_groups_user_id_6a12ed8b" ON "auth_user_groups" ("user_id")
auth_user_groups_user_id_group_id_9...		CREATE UNIQUE INDEX "auth_user_groups_user_id_group_id_94350c0c_uniq" ON "auth_user_groups" ("user_id", "grou
auth_user_user_permissions_permissio...		CREATE INDEX "auth_user_user_permissions_permission_id_1fbb5f2c" ON "auth_user_user_permissions" ("permission_i
auth_user_user_permissions_user_id_a...		CREATE INDEX "auth_user_user_permissions_user_id_a95ead1b" ON "auth_user_user_permissions" ("user_id")
auth_user_user_permissions_user_id_p...		CREATE UNIQUE INDEX "auth_user_user_permissions_user_id_permission_id_14a6b632_uniq" ON "auth_user_user_perm
core_contacto_usuario_id_cuenta_7e2...		CREATE UNIQUE INDEX "core_contacto_usuario_id_cuenta_7e24822a_uniq" ON "core_contacto" ("usuario_id", "cuenta")
core_contacto_usuario_id_e98b3e8d		CREATE INDEX "core_contacto_usuario_id_e98b3e8d" ON "core_contacto" ("usuario_id")
core_transaccion_destinatario_id_c951...		CREATE INDEX "core_transaccion_destinatario_id_c9513f87" ON "core_transaccion" ("destinatario_id")
core_transaccion_remitente_id_65f59364		CREATE INDEX "core_transaccion_remitente_id_65f59364" ON "core_transaccion" ("remitente_id")

CREATE TABLE "core_transaccion" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "tipo" varchar(20) NOT NULL, "destinatario_id" integer NULL REFERENCES "auth_user" ("id") DEFERRABLE INITIALLY DEFERRED, "fecha" datetime NOT NULL, "usuario_id" integer NOT NULL REFERENCES "auth_user" ("id") DEFERRABLE INITIALLY DEFERRED, "remitente_id" integer NULL REFERENCES "auth_user" ("id") DEFERRABLE INITIALLY DEFERRED, "monto" decimal NOT NULL)

Remote

Identity: Select an identity to connect | Upload

DBHub.io | Local | Current Database

Name | Last modified

SQL Log | Plot | Remote

settings.py | urls.py ...\alkeweb | models.py | saldo.html | views.py | urls.py ...\core

alkeweb > core > models.py > Contacto

```python
from django.db import models
from django.contrib.auth.models import User


class Contacto(models.Model):
    usuario = models.ForeignKey(User, on_delete=models.CASCADE, related_name="contactos")
    nombre = models.CharField(max_length=100)
    cuenta = models.CharField(max_length=50)

    def __str__(self):
        return f"{self.nombre} ({self.cuenta})"

class Transaccion(models.Model):
    usuario = models.ForeignKey(User, on_delete=models.CASCADE, related_name="transacciones")
    tipo = models.CharField(max_length=20, choices=[("deposito", "Depósito"), ("transferencia", "Transferencia")])
    monto = models.DecimalField(max_digits=10, decimal_places=2)
    destinatario = models.CharField(max_length=100, blank=True, null=True)
    fecha = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.tipo} - {self.monto} ({self.fecha})"
```

PROBLEMAS | Enfocar carpeta en el explorador (ctrl + clic) | NAL | PUERTOS | powershell - alkeweb

```
(venv) PS C:\Users\kgarc\OneDrive\Escritorio\alkewallet\alkeweb> python manage.py makemigrations
Migrations for 'core':
  core\migrations\0001_initial.py
    + Create model Balance
    + Create model Contacto
    + Create model Transaccion
(venv) PS C:\Users\kgarc\OneDrive\Escritorio\alkewallet\alkeweb> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, core, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
```

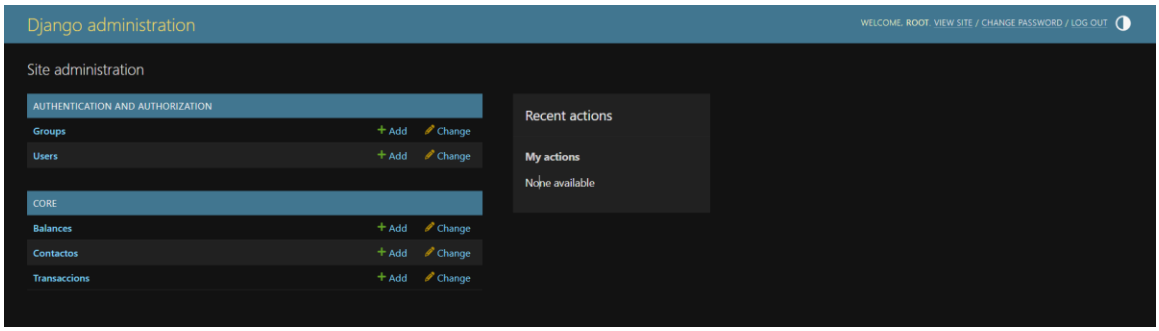
Tabulación Mueve el Foco | OVR | Lín. 9, col. 5 | Espacios: 4 | UTF-8 | CRLF | { } Python | venv (3.

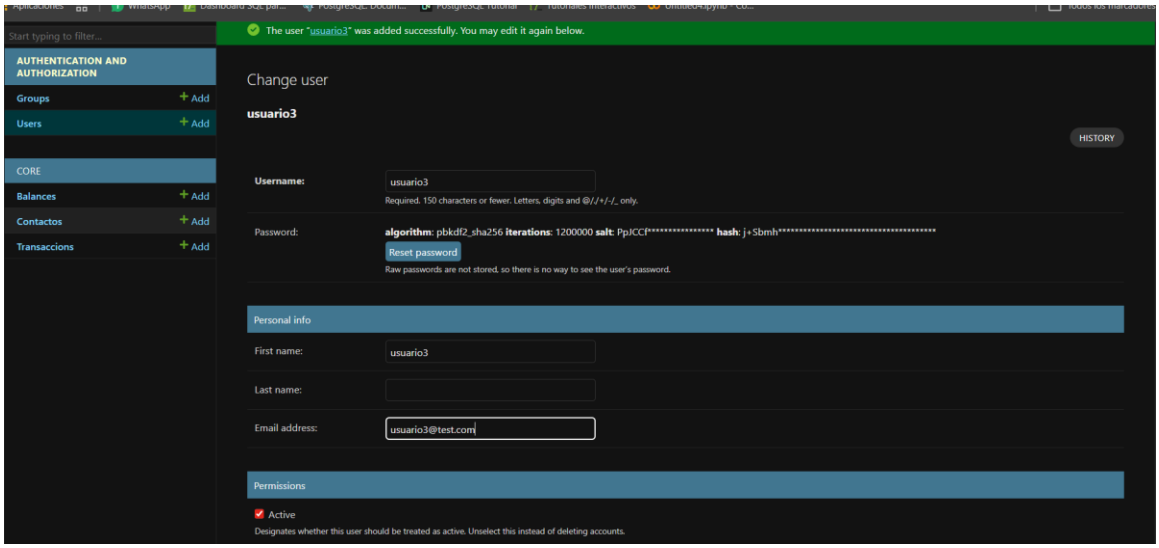
```html
<html lang="es">
<body class="d-flex flex-column min-vh-100 bg-light">
    <div class="container d-flex justify-content-center align-items-center flex-grow-1">
        <div class="card shadow p-4" style="max-width: 400px; width: 100%;">
                <label for="id_username" class="form-label">Usuario</label>
                <input type="text" name="username" id="id_username" class="form-control" required>
            </div>
            <div class="mb-3">
                <label for="id_password" class="form-label">Contraseña</label>
                <input type="password" name="password" id="id_password" class="form-control" required>
            </div>
            <button type="submit" class="btn btn-primary w-100">Ingresar</button>
        </form>

        <div class="text-center mt-3">
            <a href="{% url 'password_reset' %}" class="text-decoration-none">¿Olvidaste tu contraseña?</
        </div>
```

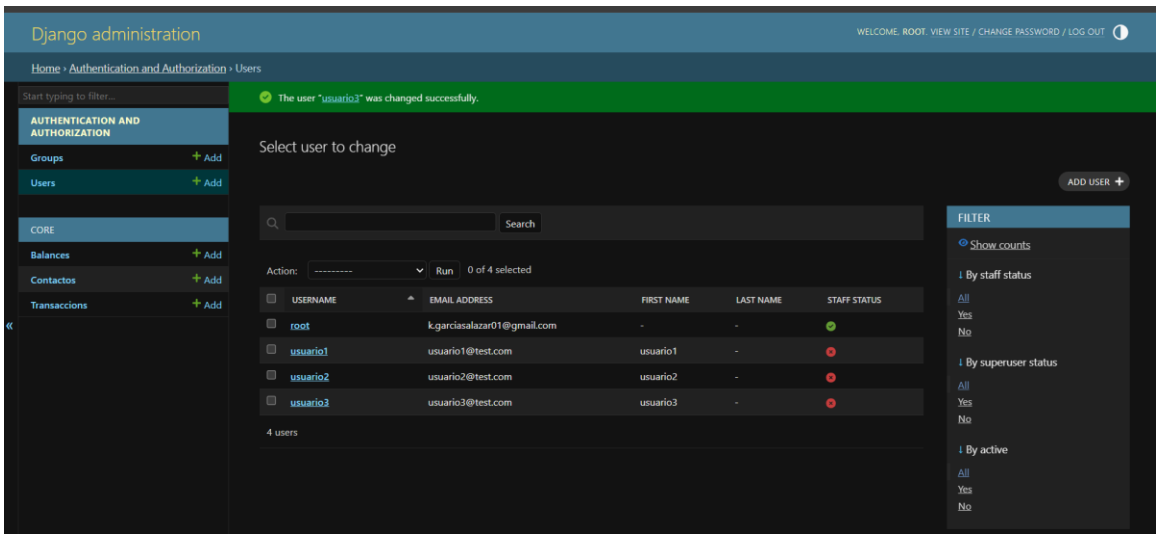
```
(venv) PS C:\Users\kgarc\OneDrive\Escritorio\alkewallet\alkeweb> python manage.py makemigrations
Migrations for 'core':
  core\migrations\0002_transaccion_remitente_alter_transaccion_destinatario_and_more.py
    + Add field remitente to transaccion
    ~ Alter field destinatario on transaccion
    ~ Alter field monto on transaccion
    ~ Alter field tipo on transaccion
    ~ Alter unique_together for contacto (1 constraint(s))
(venv) PS C:\Users\kgarc\OneDrive\Escritorio\alkewallet\alkeweb>
```

```python
from django.db import models
from django.contrib.auth.models import User


class Contacto(models.Model):
    usuario = models.ForeignKey(User, on_delete=models.CASCADE, related_name="contactos")
    nombre = models.CharField(max_length=100)
    cuenta = models.CharField(max_length=50)

    def __str__(self):
        return f"{self.nombre} ({self.cuenta})"

class Transaccion(models.Model):
    usuario = models.ForeignKey(User, on_delete=models.CASCADE, related_name="transacciones")
    tipo = models.CharField(max_length=20, choices=[("deposito", "Depósito"), ("transferencia", "Transferencia")])
    monto = models.DecimalField(max_digits=10, decimal_places=2)
    destinatario = models.CharField(max_length=100, blank=True, null=True)
    fecha = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.tipo} - {self.monto} ({self.fecha})"
```

```
(venv) PS C:\Users\kgarc\OneDrive\Escritorio\alkewallet\alkeweb> python manage.py migrate
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying core.0001_initial... OK
  Applying sessions.0001_initial... OK
(venv) PS C:\Users\kgarc\OneDrive\Escritorio\alkewallet\alkeweb> python manage.py createsuperuser
>>
Username (leave blank to use 'kgarc'): root
Email address: k.garciasalazar01@gmail.com
Password:
Password (again):
This password is too short. It must contain at least 8 characters.
This password is too common.
This password is entirely numeric.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
(venv) PS C:\Users\kgarc\OneDrive\Escritorio\alkewallet\alkeweb>
```


Django administration
WELCOME, ROOT. VIEW SITE / CHANGE PASSWORD / LOG OUT
Site administration
AUTHENTICATION AND AUTHORIZATION
Groups + Add Change
Users + Add Change
CORE
Balances + Add Change
Contactos + Add Change
Transaccions + Add Change
Recent actions
My actions
None available


Start typing to filter...
The user "usuario3" was added successfully. You may edit it again below.
AUTHENTICATION AND AUTHORIZATION
Groups + Add
Users + Add
CORE
Balances + Add
Contactos + Add
Transaccions + Add
Change user
usuario3
HISTORY
Username:
usuario3
Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.
Password:
algorithm: pbkdf2_sha256 iterations: 1200000 salt: PpJCCI********** hash: j+Sbmh**********
Reset password
Raw passwords are not stored, so there is no way to see the user's password.
Personal info
First name:
usuario3
Last name:
Email address:
usuario3@test.com
Permissions
Active
Designates whether this user should be treated as active. Unselect this instead of deleting accounts.


Django administration
WELCOME, ROOT. VIEW SITE / CHANGE PASSWORD / LOG OUT
Home › Authentication and Authorization › Users
Start typing to filter...
The user "usuario3" was changed successfully.
AUTHENTICATION AND AUTHORIZATION
Groups + Add
Users + Add
CORE
Balances + Add
Contactos + Add
Transaccions + Add
Select user to change
ADD USER +
Search
Action: --------- Run 0 of 4 selected
USERNAME
EMAIL ADDRESS
FIRST NAME
LAST NAME
STAFF STATUS
root k.garciasalazar01@gmail.com
usuario1 usuario1@test.com usuario1
usuario2 usuario2@test.com usuario2
usuario3 usuario3@test.com usuario3
4 users
FILTER
Show counts
By staff status
All
Yes
No
By superuser status
All
Yes
No
By active
All
Yes
No